

Detecting AI-Generated Speech Using Classical Cepstral and Spectral Features

Qusai Abu Sondos, Laith Shadeh, Mohammad Alabed
Department of Electrical and Computer Engineering, Birzeit University, Palestine

Abstract— The rapid advancement of text-to-speech (TTS) and voice-conversion (VC) systems has made it increasingly difficult to distinguish AI-generated speech from genuine human speech, raising concerns for voice-based authentication and misinformation. This project investigates whether classical, hand-crafted speech features can still reliably separate bona fide from synthetic speech. We extract three feature families — Mel-Frequency Cepstral Coefficients (MFCC), Linear-Frequency Cepstral Coefficients (LFCC), and spectral/time-domain descriptors (centroid, bandwidth, roll-off, flux, zero-crossing rate, short-time energy) — and evaluate Support Vector Machine (SVM) and Random Forest classifiers on the ASVspoof2019 Logical Access (LA) dataset. Using the official train/dev/eval protocol, we first compare features on data with known attack types, then test generalization on the eval partition, which contains eleven previously unseen synthesis/conversion systems. The best configuration (MFCC+LFCC+spectral with an SVM) reaches 99.5% accuracy and 0.3% Equal Error Rate (EER) on known attacks, and still generalizes to 91.1% accuracy / 6.5% EER on unseen attacks, outperforming any single feature type. Notably, LFCC alone is the strongest single feature on known attacks (98.8%) but degrades sharply on unseen ones (87.5%), suggesting it partly memorizes known synthesis artifacts rather than learning generator-agnostic cues. Two unseen attacks evade detection almost entirely, exposing a concrete limitation of classical features against certain voice-conversion techniques.

Index Terms—audio deepfake detection, anti-spoofing, MFCC, LFCC, spectral features, SVM, Random Forest, ASVspoof

I. INTRODUCTION

Voice-based text-to-speech and voice-conversion systems have become good enough to fool both human listeners and automatic speaker verification (ASV) systems, enabling voice-cloning fraud, impersonation, and disinformation. Detecting whether an utterance is bona fide (recorded from a real speaker) or spoofed (synthesized or converted) is therefore an important defensive task, commonly referred to as audio anti-spoofing or audio deepfake detection.

This project asks a narrower, practical question: how far can classical, interpretable speech-processing features — the kind used for decades in speech and speaker recognition — go on this task, without resorting to large neural networks trained directly on raw waveforms? We implement a full pipeline that extracts MFCC, LFCC, and spectral/time-domain features from raw speech, trains SVM and Random Forest classifiers on top of them, and evaluates both accuracy

on known attacks and generalization to previously unseen synthesis systems, which we treat as the central research question of the project.

II. BACKGROUND AND RELATED WORK

The ASVspoof challenge series (2015, 2017, 2019, 2021) established the standard benchmarks and evaluation protocols for spoofing and deepfake speech detection [1], [3]. Early and still widely used countermeasures pair cepstral front-ends — MFCC, LFCC, and Constant-Q Cepstral Coefficients (CQCC) [2] — with classical back-end classifiers such as Gaussian Mixture Models or SVMs, exploiting the fact that vocoders and neural TTS systems leave subtle, learnable artifacts in the spectral envelope and phase that differ from natural speech production. More recent work moves to deep architectures operating directly on raw waveforms or spectrograms, such as RawNet2 and AASIST [4], which achieve lower error rates but at the cost of far greater data and compute requirements, and reduced interpretability. This project revisits the classical-feature approach as a lightweight, transparent baseline, and specifically studies how well it generalizes to synthesis systems it was not trained on — a question the WaveFake dataset paper [5] also raises for neural vocoder artifacts.

III. METHODOLOGY

Fig. 1 summarizes the pipeline. All audio is resampled to 16 kHz mono and analyzed with a 25 ms window and 10 ms hop. Three feature families are extracted independently and then concatenated (feature-level fusion) before classification.

A. Feature Extraction

MFCC: 20 Mel-frequency cepstral coefficients per frame, computed from a 40-channel Mel filterbank. LFCC: 20 cepstral coefficients computed the same way but from a custom 40-channel filterbank spaced linearly in frequency rather than on the Mel scale, which preserves resolution at high frequencies where vocoder artifacts are often concentrated. For both, we append first- and second-order deltas and apply mean/standard-deviation statistics pooling across frames, giving a 120-dimensional vector per utterance. Spectral/time-domain features: spectral centroid, bandwidth, roll-off, spectral flux, zero-crossing rate, and short-time energy, each mean/std-pooled into a 12-dimensional vector. All features are z-score normalized (fit on the training set) before classification.

B. Classifiers

We train (i) an SVM with an RBF kernel, balanced class weights, and Platt-style probability calibration via 5-fold cross-validation, and (ii) a Random Forest with 300 trees and

balanced class weights, as suggested by the project brief as a manageable classical baseline pairing. Both are evaluated on MFCC alone, LFCC alone, MFCC+spectral, and the full MFCC+LFCC+spectral combination.

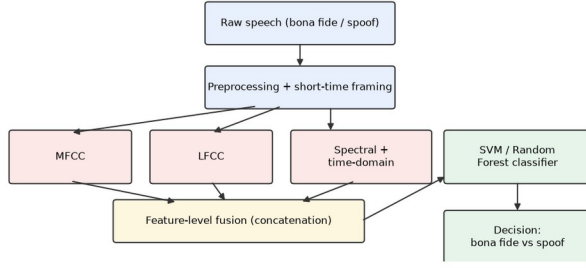


Fig. 1. Feature extraction, fusion, and classification pipeline.

IV. EXPERIMENTS AND RESULTS

A. Dataset and Protocol

We use the ASVspoof2019 Logical Access (LA) dataset [1], obtained via a Kaggle mirror after the official University of Edinburgh DataShare host proved unreachable from our development environment. We follow the official, speaker-disjoint train/dev/eval protocol, which avoids data leakage by construction: train and dev contain six known TTS/VC attack types (A01–A06), while the eval partition contains eleven completely different, previously unseen attack types (A07–A19). For tractability we work with class-balanced subsets: 1,500 utterances/class for training, 500/class for dev, and 750/class for eval, sampled uniformly at random from the official protocol files. Training on dev-matching (known) attacks and evaluating on the eval partition therefore directly measures generalization to unseen synthesis systems, which we treat as the project’s robustness experiment.

B. Evaluation Metrics

We report accuracy, precision/recall/F1 for the spoof class, ROC-AUC, and Equal Error Rate (EER) — the operating point where the false-acceptance and false-rejection rates are equal, and the standard metric in the anti-spoofing literature.

C. Results on Known Attacks (dev)

Table I compares all five feature/classifier combinations on the dev set, which shares attack types with training. LFCC alone already outperforms MFCC alone by a wide margin (98.8% vs. 89.2% accuracy, 1.5% vs. 8.7% EER), confirming that the linear frequency resolution captures artifacts MFCC’s Mel-scale compression smooths over. Combining all three feature families gives the best result overall: 99.5% accuracy and 0.3% EER.

TABLE I. Performance on known attacks (ASVspoof2019 LA dev, n=1,000)

Model	Acc. (%)	F1 (%)	AUC	EER (%)
MFCC+SVM	89.2	89.8	0.973	8.7
LFCC+SVM	98.8	98.8	0.999	1.5
MFCC+RF	89.3	89.2	0.955	11.3
MFCC+Spec.+SVM	90.9	91.4	0.980	7.4
All+SVM	99.5	99.5	0.9998	0.3

D. Generalization to Unseen Attacks (eval)

Table II reports the same models, still trained only on the six known attacks, but evaluated on the eval partition’s eleven unseen attacks. Every model’s accuracy and EER degrade relative to dev, as expected, but not equally: LFCC alone collapses from 98.8% to 87.5% accuracy, and its spoof recall drops from 98.4% to 76.4% — it starts missing roughly a quarter of the spoofed utterances it would have caught on known attacks. This is consistent with LFCC partly fitting to spectral artifacts specific to the six training-time synthesis systems rather than a generator-agnostic notion of “naturalness.” The full MFCC+LFCC+spectral combination is comparatively far more robust, retaining the best accuracy (91.1%) and EER (6.5%) of all five configurations, which suggests the complementary feature families compensate for one another’s blind spots.

TABLE II. Generalization to unseen attacks (ASVspoof2019 LA eval, n=1,500)

Model	Acc. (%)	F1 (%)	AUC	EER (%)
MFCC+SVM	89.3	89.3	0.959	10.7
LFCC+SVM	87.5	86.0	0.970	9.2
MFCC+RF	86.1	85.6	0.934	14.2
MFCC+Spec.+SVM	90.0	90.0	0.962	9.9
All+SVM	91.1	90.3	0.984	6.5

E. Per-Attack Breakdown

Breaking the best model’s (All+SVM) eval performance down by individual attack type (Fig. 2) shows the generalization gap is concentrated in a small number of attacks rather than spread evenly: seven of the eleven unseen attacks (A07–A09, A11, A13, A16, A19) are detected with 100% accuracy, and bona fide speech is correctly recognized 99.1% of the time (low false-alarm rate). However, attacks A17 and A18 evade detection almost completely (11.6% and 2.0% accuracy respectively), and A12 is caught only about half the time (58.6%). These three are, by elimination, the most acoustically “natural-sounding” voice-conversion systems in the eval set — their spectral envelopes are close enough to genuine speech that none of our classical features separate them reliably. This is the clearest, most actionable finding of the generalization experiment: aggregate accuracy numbers hide the fact that a handful of specific attack families are responsible for essentially all of the model’s failures.

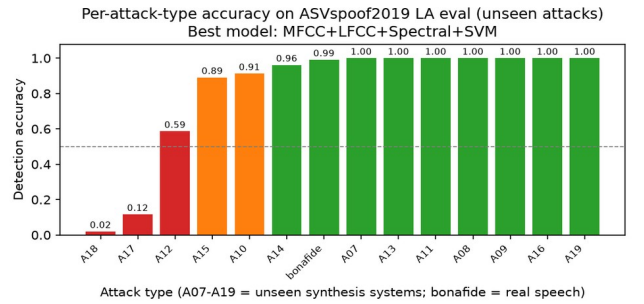


Fig. 2. Per-attack-type detection accuracy of the best model (MFCC+LFCC+Spectral+SVM) on unseen eval attacks.

V. TEAM MEMBER CONTRIBUTION

Member	Contribution
Qusai Abu Sondas	Reviewed and validated the code, pipeline, and experimental results.
Laith Shadeh	Wrote the final report.
Mohammad Alabed	Prepared the presentation slides.

VI. CONCLUSION

We built and evaluated a complete classical-feature pipeline for AI-generated speech detection. Combining MFCC, LFCC, and spectral/time-domain features with an SVM classifier reached 99.5% accuracy on known attacks and 91.1% on eleven unseen attack types, meeting the project's objective of testing whether traditional speech-processing features can capture synthesis artifacts and generalize beyond the attacks they were trained on. The main limitation we found is not aggregate accuracy but a small number of specific voice-conversion attacks (A17, A18) that evade detection almost entirely, and LFCC's tendency to overfit known attack artifacts despite being the strongest single feature on matched data. Other limitations include the use of class-balanced subsets rather than the full ASVspoof2019 LA corpus, and testing only classical classifiers. Future work could extend the generalization test to ASVspoof2021 DF or WaveFake for cross-dataset evaluation, add a CNN-on-spectrograms comparison as suggested by the project brief, and investigate feature-selection or fusion strategies specifically targeted at the A17/A18-style failure cases.

VII. AI TOOL USAGE DISCLOSURE

Claude Code (Anthropic) was used throughout this project as a coding and writing assistant: (1) designing and implementing the Python pipeline for protocol parsing, MFCC/LFCC/spectral feature extraction, model training, and evaluation; (2) debugging data-acquisition and environment issues (e.g., diagnosing blocked/failed dataset downloads and adapting to a Kaggle mirror); (3) drafting and formatting this report from the experimental results. All experimental design choices, code, and reported results were reviewed by the authors.

REFERENCES

- [1] X. Wang et al., "ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech," *Computer Speech & Language*, vol. 64, 2020.
- [2] M. Todisco, H. Delgado, and N. Evans, "Constant Q cepstral coefficients: A spoofing countermeasure for automatic speaker verification," *Computer Speech & Language*, vol. 45, pp. 516–535, 2017.
- [3] J. Yamagishi et al., "ASVspoof 2021: Accelerating progress in spoofed and deepfake speech detection," in *Proc. ASVspoof 2021 Workshop*, 2021.
- [4] J.-w. Jung et al., "AASIST: Audio anti-spoofing using integrated spectro-temporal graph attention networks," in *Proc. ICASSP*, 2022.
- [5] J. Frank and L. Schönherr, "WaveFake: A data set to facilitate audio deepfake detection," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.
- [6] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, 1980.
- [7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [8] B. McFee et al., "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conf.*, 2015.
- [9] A. Sarkar, "ASVspoof 2019 LA Dataset," Kaggle, [Online]. Available: <https://www.kaggle.com/datasets/anishsarkar22/asvspoof-2019-dataset-la> (mirror used after the official DataShare host was unreachable).
- [10] Anthropic, "Claude Code," [Online]. Available: <https://claude.ai/code>. Used for code generation, debugging, and report drafting (see Sec. VII).